

Linux Privilege Escalation: CVE-2016-5195 ("Dirty COW")

A critical memory management vulnerability

Theodor Negrescu

Master's Student, University of Bucharest, Romania

2026-02-03

Abstract

CVE-2016-5195, also known as "Dirty COW", was a critical vulnerability in the Linux kernel which allowed users to elevate their privileges to root by editing read-only files such as `setuid` binaries [1, 2].

This was caused by a race condition in the kernel's memory management code [3], involving the interaction between `madvise(MADV_DONTNEED)` and writes to copy-on-write (COW) memory through `/proc/self/mem/`.

This interaction allowed an attacker to write to shared memory pages in the page cache for memory-mapped read-only files.

Writing to the page cache allows an attacker to permanently modify any files they have read access to, since the page cache is written back to disk eventually.

This incident provides valuable lessons for kernel security, both for secure concurrent kernel programming in general, and for memory management subsystems in particular, where subtle bugs can compromise crucial isolation boundaries.

1 Introduction

Copy-on-write (COW) is an optimization used by the Linux kernel to share memory between processes, even though the processes may eventually write to that memory and expect to have a private copy.

It is extremely beneficial to be able to share memory even when it is supposed to have private writable semantics, since it enables sharing memory between forked processes, and also sharing the page cache when memory-mapping files.

The fundamental insight behind COW is that a private writable mapping can still be backed by shared memory, as long as an actual private copy is created when a write occurs.

By marking the hardware page table entry read-only, the kernel can transparently intercept write attempts through page faults and copy the page on-demand.

Ordinarily, this is completely invisible to user-space processes, besides the latency when writing to COW pages for the first time.

Processes are given the abstraction of a private writable mapping, which is implemented as a shared copy until the first write.

Because COW is used to share the page cache between the kernel and untrusted user-space processes, it is critical for security that processes are never allowed to write to shared pages.

Dirty COW breached this protection by exploiting a race condition in the `/proc/self/mem` indirect write handling code, which causes the kernel to incorrectly write to the shared page if the private copy is discarded by `madvise` at the right time.

While not a traditional memory corruption bug (it does not involve a buffer overflow or other standard illegal memory access issue), it still achieves the same ultimate outcome as a typical buffer overflow exploit - writing arbitrary data to memory beyond the attacker's intended privilege level.

Because the exploit operates at the kernel memory management level, ordinary memory protection mechanisms are bypassed completely.

2 Technical Background

The Dirty COW exploit does not target the primary page-fault handling path through direct writes to COW pages, but instead uses indirect writes through `/proc/self/mem` or `ptrace`.

To understand why using `/proc/self/mem` is necessary, we must understand how COW memory ordinarily works in Linux, the intended behavior when mapping read-only files, and how `/proc/self/mem` deviates from this behavior in an attempt to provide a more convenient debugging interface.

It is not actually forbidden to create a private writable mapping of a file you do not have write access to, however this does not let you modify the underlying file. You simply get copy-on-write pages which get converted to private copies when you write to them and are never written to disk.

Indirect writes deviate only slightly from standard behavior. They allow writing to read-only mappings as if they were writable, but this is still supposed to have the ordinary copy-on-write semantics. There should still be a private copy created on write which is not written back to disk.

The reasoning behind this is that `/proc/self/mem` and `ptrace` are primarily intended as debugging interfaces, and there are sometimes reasons to modify read-only memory, such as to patch code in a running process. Requiring manual modification of the mappings would be inconvenient and error-prone.

It is this debugging feature which was exploited by Dirty COW.

While handling writes to `/proc/self/mem`, `__get_user_pages` is called, which is supposed to pin a process's page into memory for use by the kernel.

That function involves searching for pages as follows:

1. Try to find a page by walking the process's page table using the `follow_page_mask(..., flags, ...)` function.
2. If the page was not found, trigger the page fault handler using `faultin_page(..., &flags, ...)`. If this succeeds, go to step 1. (Error handling if the page cannot be faulted in is not relevant for Dirty COW.)

3. If the page is found, break the loop and return the page.

The `faultin_page` function brings a page into memory and pins it by incrementing a reference count.

It also has some special logic for supporting writes to read-only COW pages, enabled by the `FOLL_FORCE` flag set by `/proc/self/mem` and `ptrace`.

If COW handling was successfully performed and the caller is trying to write to a read-only page, it will unset the write bit `FOLL_WRITE` from the caller's flags, which are passed by reference.

Passing control flags by reference in this way and modifying them from within utility functions is actually a common pattern in the Linux memory management subsystem.

The intended behavior is that the next iteration of the retry loop in `__get_user_pages` will call `follow_page_mask` without the `FOLL_WRITE` set, locating the newly created read-only private mapping in the process's page table.

However, if the private page is discarded between `faultin_page` creating it and the next call to `follow_page_mask`, then there is nothing for `follow_page_mask` to find, and `faultin_page` is called again with the modified flags.

Since `FOLL_WRITE` is now unset, `faultin_page` sees that the caller just wants to read from a file-backed mapping, so it doesn't see any need to create a private copy. It simply returns the kernel's internal writable mapping from the page cache, corresponding to the privileged file. Then the next `follow_page_mask` call will find that page, and it is used for the write operation.

To trigger this chain of events, the attacker needs a way to cause the private copy to be discarded at the right time.

The `madvise` function provides exactly a way to do this with the `MADV_DONTNEED` flag, which advises the kernel that a memory range will not be needed soon, so the kernel should discard it from memory and page it back in when needed to reduce memory pressure.

Writing to `/proc/self/mem` and calling `madvise(MADV_DONTNEED)` in separate threads in tight loops is virtually guaranteed to successfully perform the exploit in seconds.

In short, the exploit proceeds as follows:

1. Thread A maps a sensitive system file read-only with `mmap(MAP_PRIVATE)`.
2. Thread A writes to the mapping through `/proc/self/mem`. This should only result in a private copy being modified.
3. The kernel calls `__get_user_pages` to find and pin the page for writing.
4. `__get_user_pages`'s first call to `follow_page_mask` fails, so it calls `faultin_page` to handle the page fault.
5. `faultin_page` creates the private copy and clears `FOLL_WRITE` in the flags argument.
6. Thread B's `madvise(MADV_DONTNEED)` call discards the private copy created in step 5. This is where the race condition happens.
7. The second `follow_page_mask` call fails.
8. `faultin_page` again is called with the modified flags, which were not supposed to end up used for a call to `faultin_page`.
9. `faultin_page` sees that `FOLL_WRITE` is unset and thinks it is safe to use the shared page from the page cache, but actually it itself had unset `FOLL_WRITE` in step 5.
10. The third call to `follow_page_mask` succeeds and returns the shared page from the page cache.
11. The kernel's `/proc/self/mem` handler unintentionally writes to the page cache. The attacker has successfully silently modified the underlying file.
12. If the race condition failed, `madvise` still discarded the private copy, just not at exactly the right time, so the exploit will keep creating and discarding private copies until it succeeds.

3 Exploitation and Impact

Dirty COW was being exploited in the wild when discovered by security researcher Phil Oester while he was investigating a packet capture from a compromised web server. [2]

Because the exploit requires no privileges besides basic system calls which almost nobody restricts, it

can escalate almost any remote code execution to full system compromise.

It was found to affect all kernel versions from 2007 to October 2016, with applications such as rooting any Android phone at the time.

Linus Torvalds released a patch on 20 October 2016 [4], and some distributions provided live patches to allow quick updating without downtime. He notes that he had attempted to fix this bug over a decade earlier, but the fix was reverted for causing breakages.

Because the exploit can modify system files with no trace, most mitigations and intrusion detection systems are ineffective.

It is not necessary to target `setuid` binaries in particular; any sensitive read-only file can be modified, such as the password hash database, many configuration files, cron jobs, or binaries used by privileged services.

Modifying a program and just waiting for it to be run during normal system operation is effectively undetectable prior to payload execution. Intrusion detection systems could still notice the payload execution itself, which is true of almost any exploit, but is often easy to evade in practice.

4 The Fix

Linus Torvalds's patch implements a straightforward fix for this vulnerability: setting a separate `FOLL_COW` bit in the flags argument to indicate that the caller intends to write to a COW page, but only if it is a private copy.

`faultin_page` function still modifies the caller's flags, but now it does so by setting `FOLL_COW`, instead of clearing `FOLL_WRITE`. This provides context for `follow_page_mask` without removing information.

If the private copy is discarded, the next call to `faultin_page` will not fetch the shared page, because `FOLL_WRITE` is still set. The loop will just keep retrying until `madvise` doesn't manage to discard the copy.

There is no longer any execution path which leads to returning the shared page to the `/proc/self/mem` handler.

The diff which implements the fix is just a few lines, a macro definition for `FOLL_COW`, and a helper function. `can_follow_write_pte` checks

whether a page is either writable, or `FOLL_COW` and `FOLL_FORCE` are set and the page is a private copy (i.e. is "dirty").

5 Similar Vulnerability

A second vulnerability, CVE-2017-1000405 ("Huge Dirty COW"), was discovered in 2017 [5].

It exploits a bug in the handling of Transparent Huge Pages (THP). Although Linus had also attempted to patch this, he had missed a slight difference which allowed the bug to persist.

THP is a feature to automatically compact adjacent pages into hardware-supported larger pages to reduce pressure on the translation lookaside buffer, the internal cache of the MMU.

Luckily, this was much less severe, because regular file mappings do not use THP.

It does allow writing to the shared huge zero page and to sealed shared memory segments (memory-backed `shmem` file descriptors, not disk-backed files).

The huge zero page is a shared page filled with zeros returned when allocating memory, since Linux uses overcommit for memory allocation. Modifying it could affect programs relying on zero-initialized memory, but it is replaced with a freshly zeroed page after being written to, which heavily limits its impact.

Sealing is a mitigation where a process can give up rights to modify or extend a shared memory segment, to prevent itself from being exploited to attack other processes sharing the same segment.

While shared memory is used for many purposes, and defeating sealing could be very useful for attacking multi-process applications (for example: browsers with tab isolation), this is still much less dangerous than the original Dirty COW vulnerability.

This vulnerability was discovered by analysis and there was no known exploitation in the wild.

6 Conclusion

Many lessons can be learned from the Dirty COW vulnerability.

Keep logic explicit.

Clearing `FOLL_WRITE` from inside `faultin_page` to communicate "it is okay to write to a private copy even if marked read-only" is hard to reason about and led to an exploitable bug.

There are legitimate reasons Linux uses the flags argument as a bidirectional communication channel, but clearing a general flag like `FOLL_WRITE` to communicate a subtle condition was a bad idea.

The patch still modifies flags inside `faultin_page`, but now it sets `FOLL_COW` which explicitly communicates that a COW private copy has been created and future calls to `follow_page_mask` know exactly what is happening. The new helper function `can_follow_write_pte` explicitly checks if the page is "dirty" (meaning modified, i.e. a private copy).

This is much cleaner than just clearing `FOLL_WRITE`, which had unintended consequences when the retry loop ended up calling `faultin_page` again. The new bit has no effect on `faultin_page`.

Secure concurrent code is hard.

The original code would have been completely safe, though still a bit hacky, if the process's page table had been kept write-locked during the entire retry loop. But that would block other threads from handling page faults in a multi-threaded process, and most faults don't even involve COW.

Performance in concurrency tends to require more complex code. Unfortunately the kernel needs both high performance and high security.

Debugging features are an attack surface.

There are rarely reasons to write to memory through `/proc/self/mem` or `ptrace` in production code.

Convenient features like writing to read-only mappings, even when they only slightly increase code complexity, are an attack surface.

Security is never perfect.

Although in theory it's possible to build provably secure systems, such as the seL4 microkernel, that's usually not practical.

Linux today probably has undiscovered vulnerabilities similar in severity to Dirty COW.

The lesson for large organizations is to always use defense in depth, and to try to be prepared even for root-level compromises.

For individuals, basic measures like backups and 2FA can limit the damage of most attacks.

References

- [1] NIST. CVE-2016-5195 Detail, November 2016. URL <https://nvd.nist.gov/vuln/detail/cve-2016-5195>. Accessed: 2026-02-03.
- [2] Dan Goodin. "Most serious" Linux privilege escalation bug ever is under active exploit. *Ars Technica*, October 2016. URL <https://arstechnica.com/information-technology/2016/10/most-serious-linux-privilege-escalation-bug-ever-is-under-active-exploit/>. Accessed: 2026-02-03.
- [3] Chao-tic. Dirty COW and why lying is bad even if you are the Linux kernel, May 2017. URL <https://chao-tic.github.io/blog/2017/05/24/dirty-cow>. Accessed: 2026-02-03.
- [4] Linus Torvalds. mm: remove gup_flags FOLL_WRITE games from __get_user_pages(). Linux Kernel Mailing List, October 2016. URL <https://lkml.org/lkml/2016/10/19/860>. Accessed: 2026-02-03.
- [5] Eylon Ben Yaakov. "Huge Dirty COW" (CVE-2017-1000405). The incomplete Dirty COW patch. Medium, November 2017. URL <https://medium.com/bindecy/huge-dirty-cow-cve-2017-1000405-110eca132de0>. Accessed: 2026-02-03.